

18-799: Applied Computer Vision

Spring 2026

Artificial Neural Network

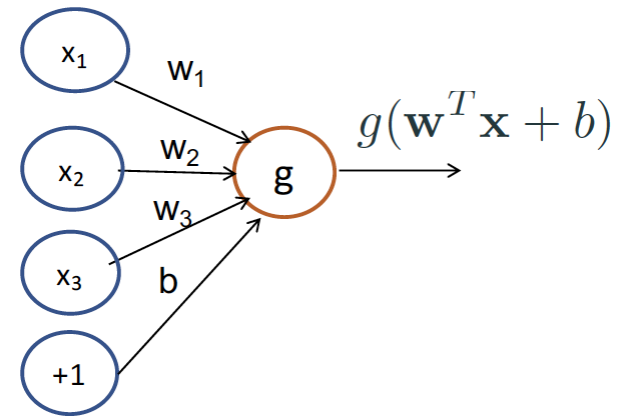
Objectives

- In this class we will learn:
 - What is an artificial neuron
 - The difference between a perceptron and linear models
 - Artificial Neural Network
 - Convolution Neural Network

Artificial Neuron

The mathematical model

- Each input x_i is multiplied by weight w_i
- Add a +1 input neuron, which is multiplied by the bias b
- Apply a non-linear function g to the weighted combination of the inputs, $\mathbf{w}^T \mathbf{x} + b$
- Different candidates for g : heaviside function, sigmoid, tanh, rectified linear unit, etc.

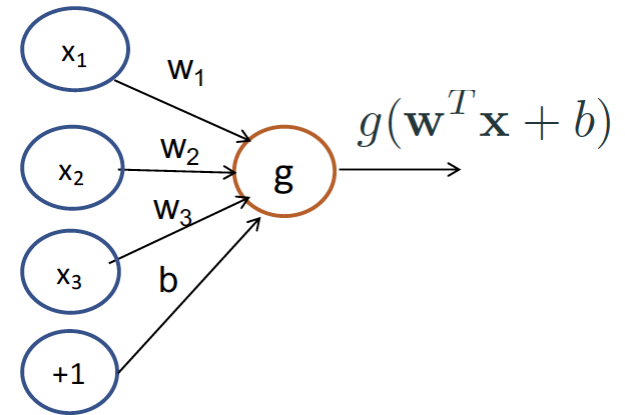


Single Artificial Neuron

Artificial Neuron

Pass inputs through a “network” of neurons to obtain outputs.

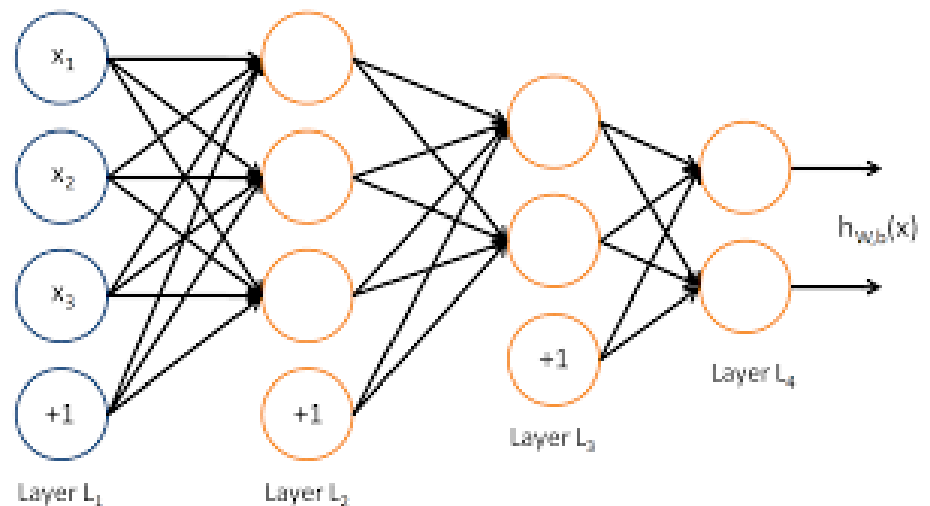
- Neural networks are very good at handling large-scale data.
- They can learn very complex relationships.
- Requires careful configuration: what does this network look like?



Single Artificial Neuron

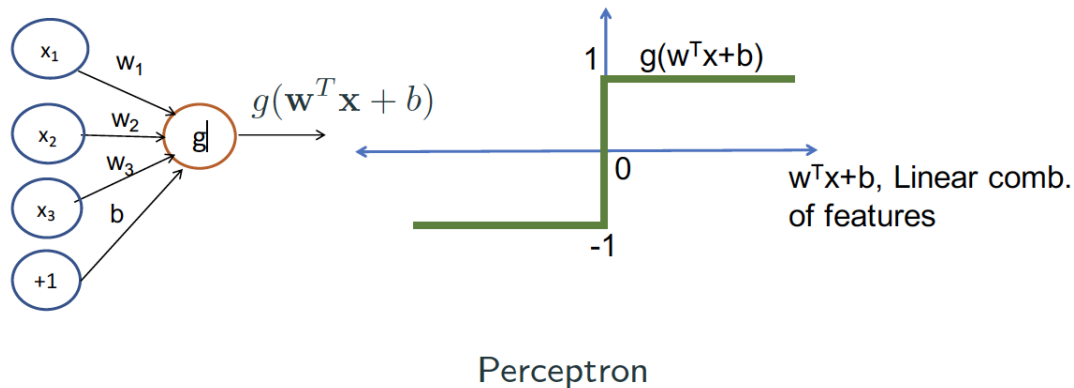
Artificial Neuron

- Each neuron is sometimes called a “node” or “unit” in the network.
- We group functions into “layers” depending on how many neurons their inputs have passed through since the original inputs.



Perceptron

- The perceptron is a single-unit neural network with the Heaviside activation function, $\text{sign}(x)$.
- It considers a linear binary classification problem to distinguish between two classes $\{-1, +1\}$.
- Assign label $\text{sign}(\mathbf{w}^T \mathbf{x} + b)$ to a new sample



How can we learn the weights?

How can we change $\mathbf{w}$ such that

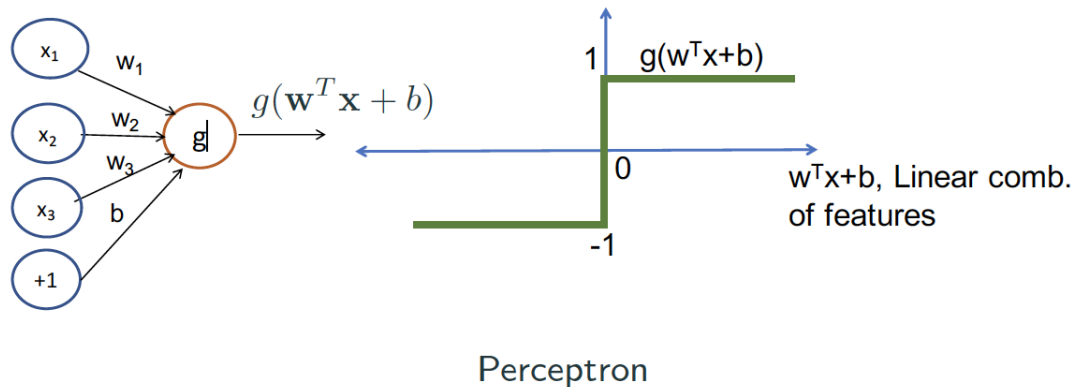
$$y_n = \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$$

for a given training example $(\mathbf{x}_n, y_n)$?

Two cases

- If $y_n = \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$, do nothing.
- If $y_n \neq \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$,

$$\mathbf{w}^{\text{NEW}} \leftarrow \mathbf{w}^{\text{OLD}} + y_n \mathbf{x}_n$$



Would it work?

- If $y_n \neq \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$, then

$$y_n(\mathbf{w}^\top \mathbf{x}_n) < 0$$

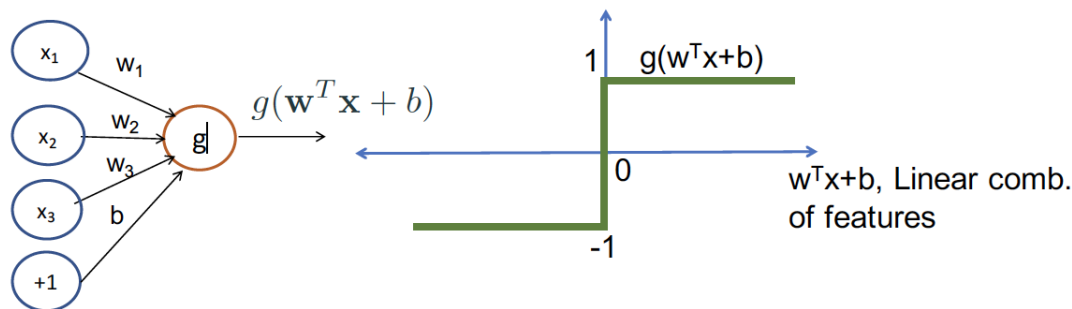
- What would happen if we change to new $\mathbf{w}^{\text{NEW}} = \mathbf{w} + y_n \mathbf{x}_n$?

$$y_n[(\mathbf{w} + y_n \mathbf{x}_n)^\top \mathbf{x}_n] = y_n \mathbf{w}^\top \mathbf{x}_n + y_n^2 \mathbf{x}_n^\top \mathbf{x}_n$$

- We are adding a positive number, so it is possible that

$$y_n(\mathbf{w}^{\text{NEW}^\top} \mathbf{x}_n) > 0$$

- i.e., we are more likely to classify correctly



Perceptron

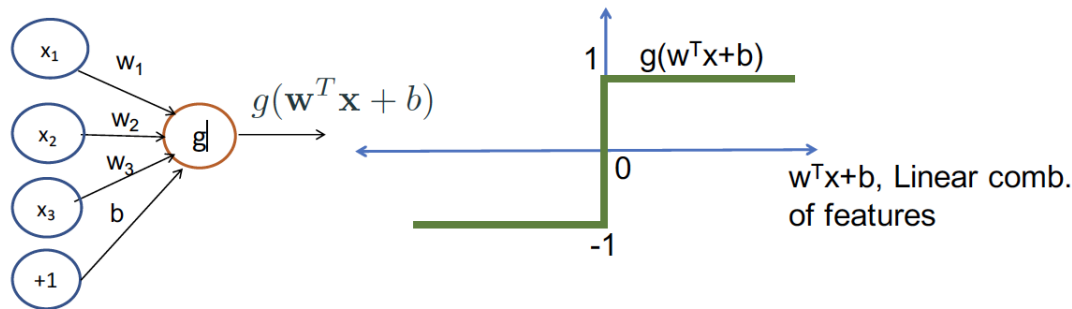
Training

Iteratively solving one case at a time

- REPEAT
- Pick a data point $\mathbf{x}_n$
- Make a prediction $y = \text{sign}(\mathbf{w}^T \mathbf{x}_n)$ using the current $\mathbf{w}$.
- If $y = y_n$, do nothing.
Else,

$$\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$$

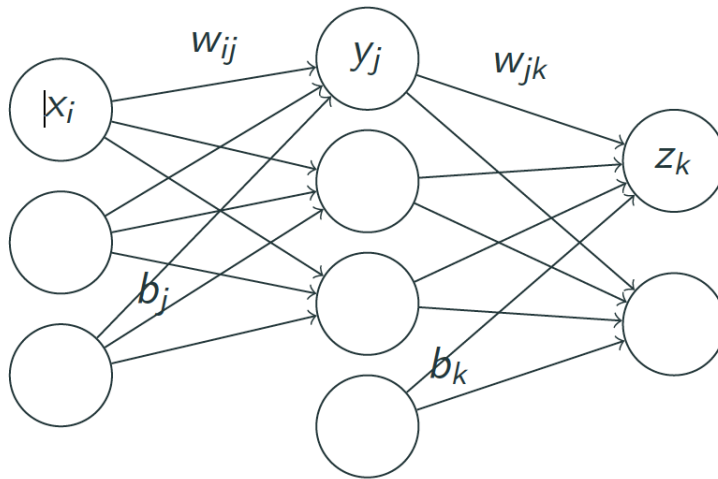
- UNTIL converged.



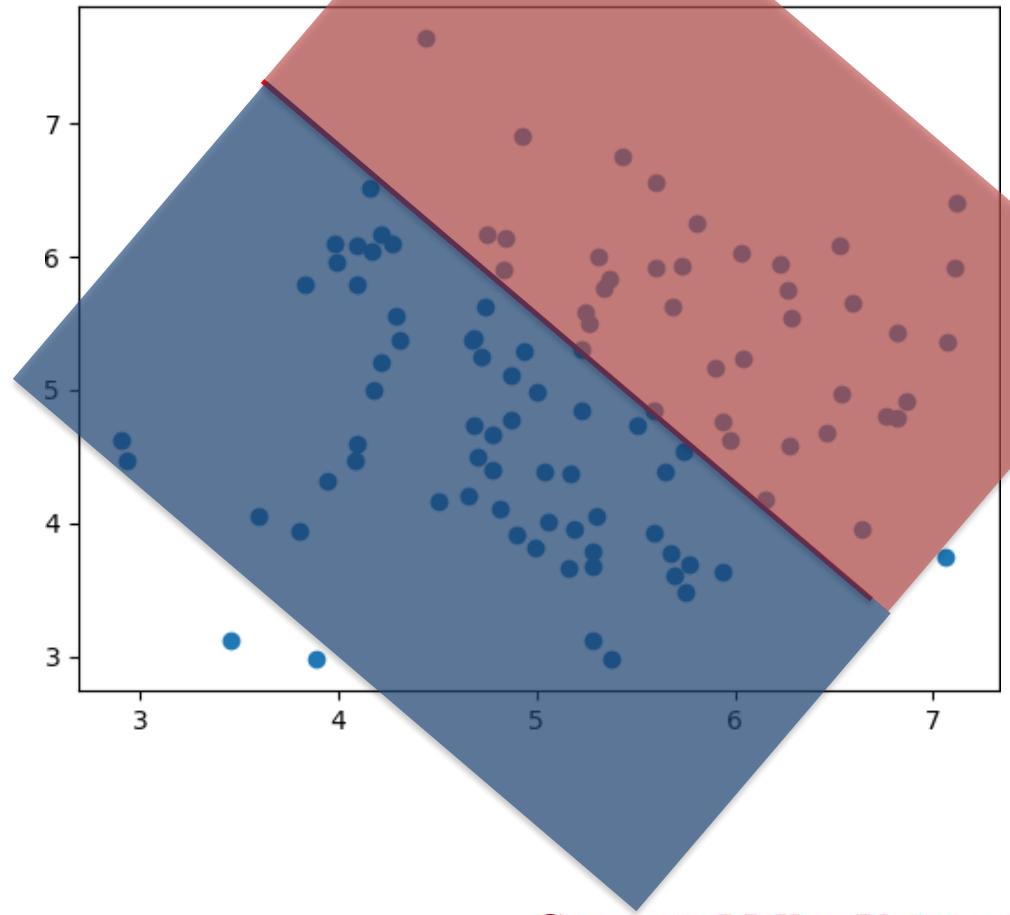
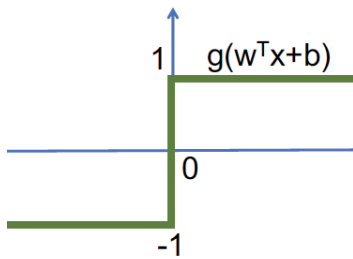
Perceptron

Multi-layer Neural Network

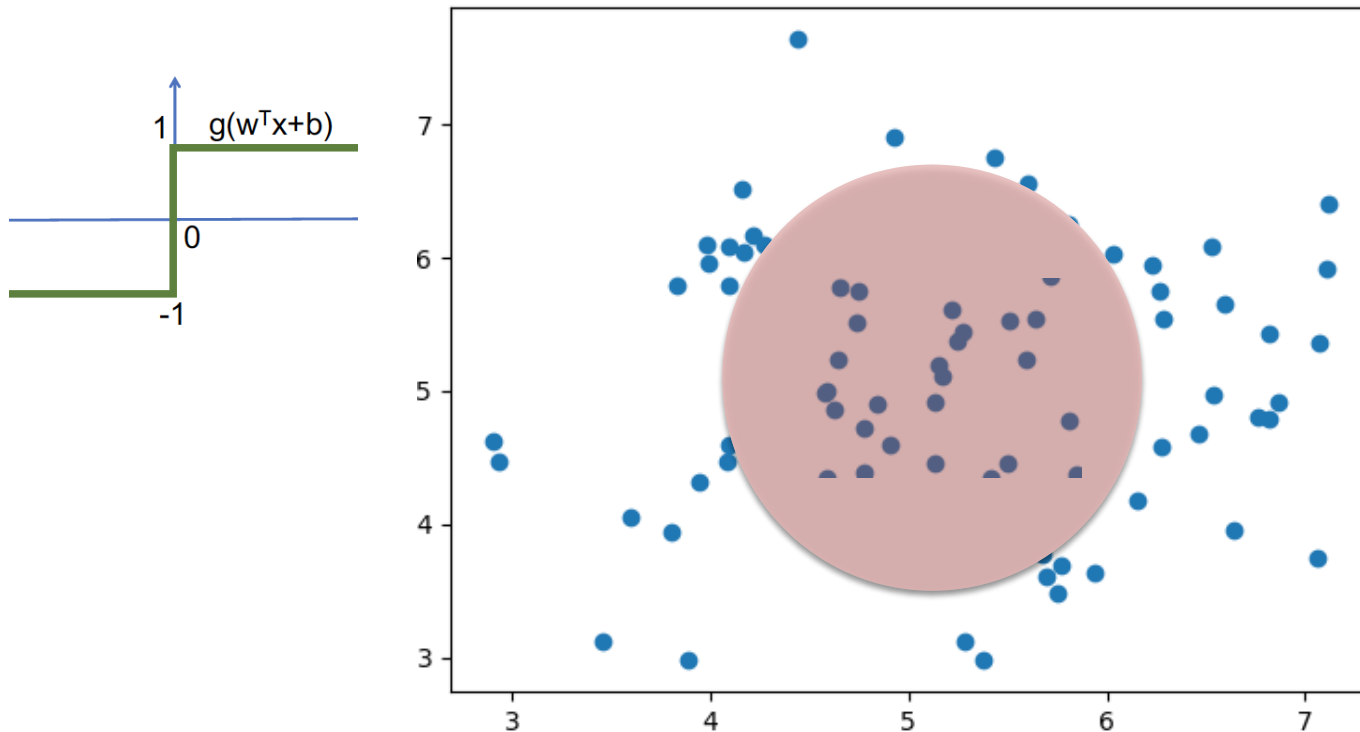
- w_{ij} : weights connecting node i in layer $(\ell - 1)$ to node j in layer ℓ .
- b_j, b_k : bias for nodes j and k .
- u_j, u_k : inputs to nodes j and k (where $u_j = b_j + \sum_i x_i w_{ij}$).
- g_j, g_k : activation function for node j (applied to u_j) and node k .
- $y_j = g_j(u_j), z_k = g_k(u_k)$: output/activation of nodes j and k .
- t_k : target value for node k in the output layer.



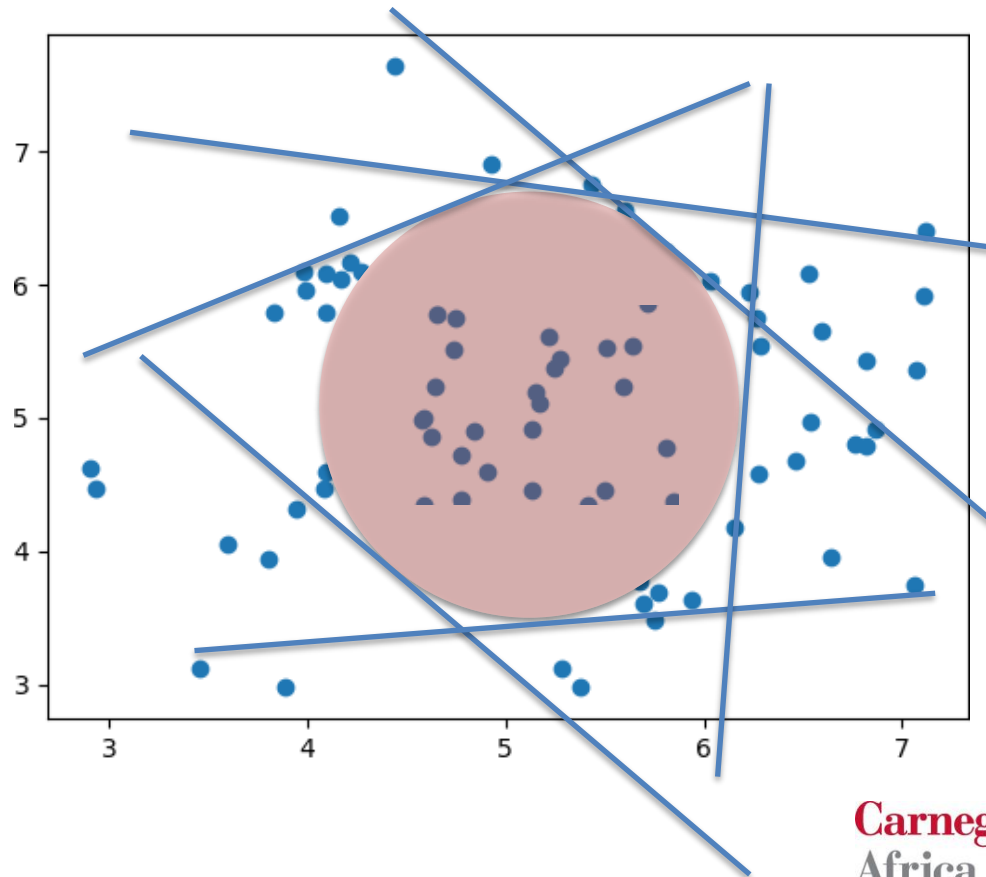
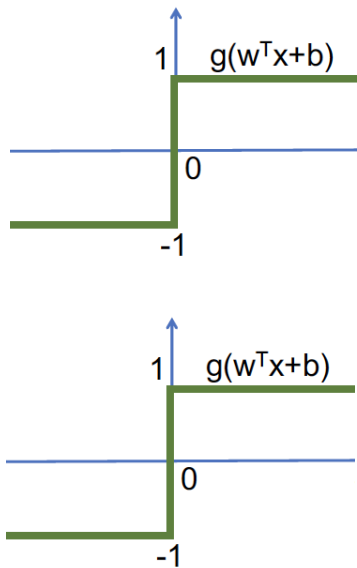
Recall: Perceptron



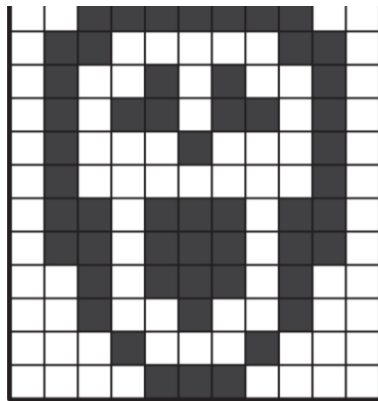
Recall: Perceptron



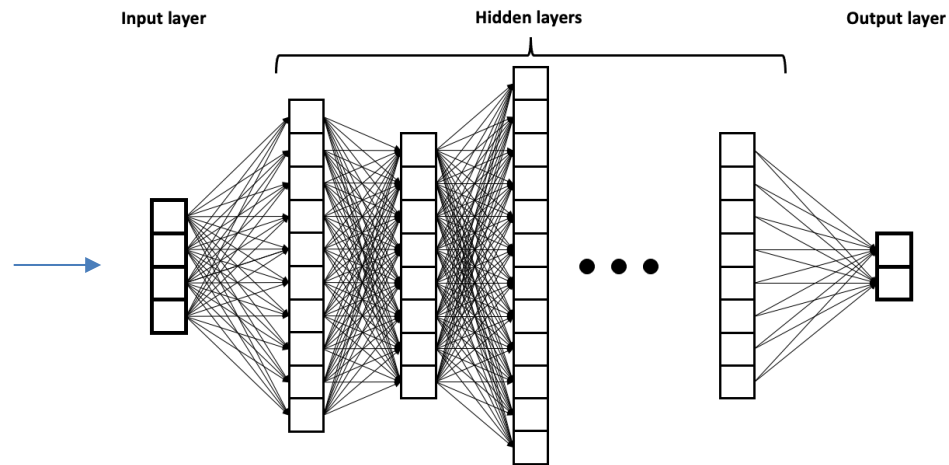
Recall: Perceptron



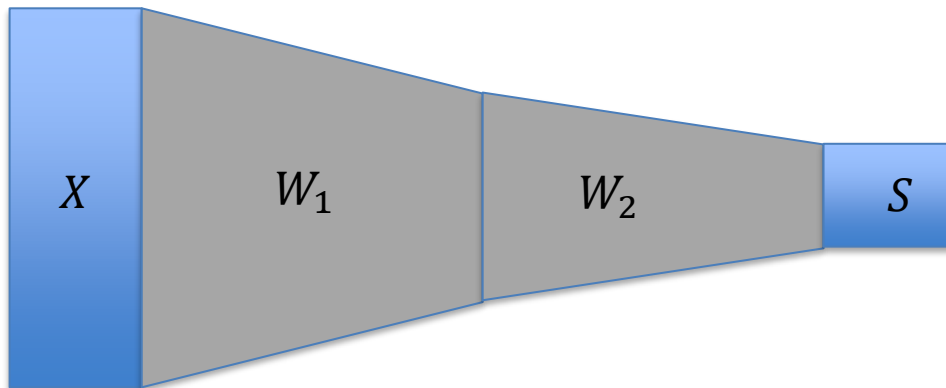
Artificial Neural Network



Source: <https://www.vecteezy.com/free-vector/game>

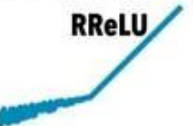
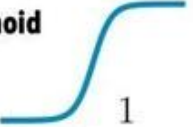
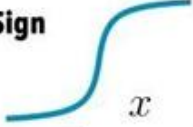
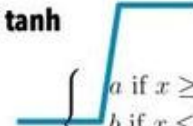
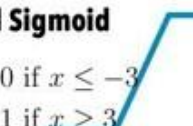
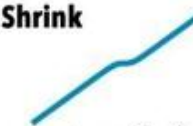
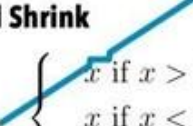


-
- Linear Models: $f = Wx$
 - Two layer Network: $f = W_2x' = W_2(W_1x) = \mathbf{W'}x}$ with $\mathbf{W'} = W_2W_1$



Activation functions

Neural Network Activation Functions: a small subset!

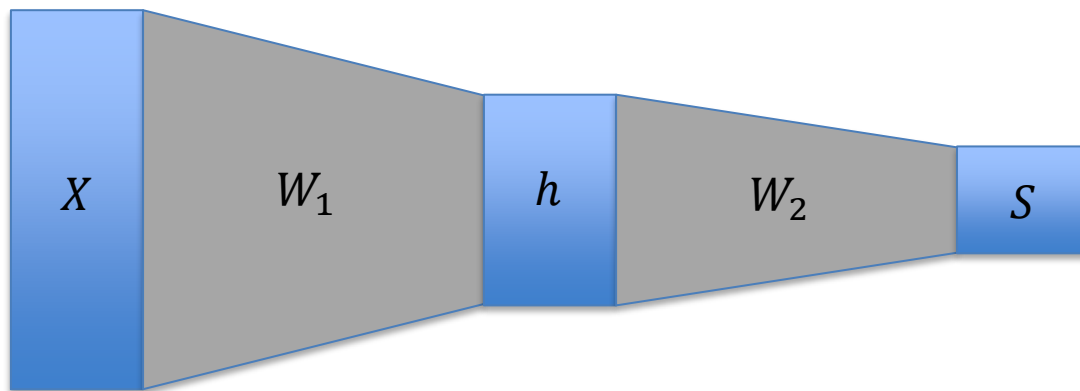
ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

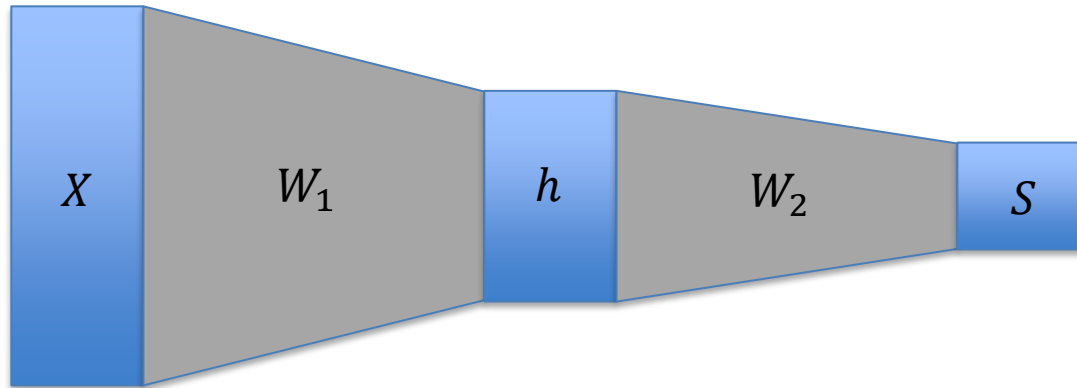
<https://www.linkedin.com/pulse/activation-functions-heba-al-haddad>

-
- Linear Models: $f = Wx$
 - Two layer Network: $f = W_2x' = W_2g(W_1x) = W_2 \max(0, W_1x)$
 - or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1x))$$

$$W_3 \in \mathbb{R}^{C \times H_2} \quad W_2 \in \mathbb{R}^{H_2 \times H_1} \quad W_1 \in \mathbb{R}^{H_1 \times D} \quad x \in \mathbb{R}^D$$

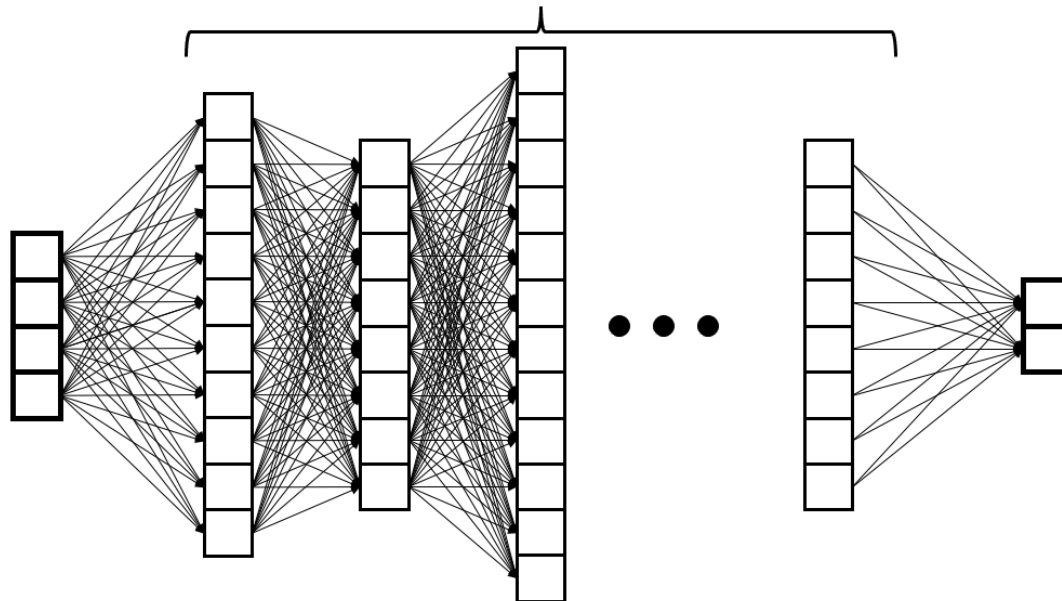




Input layer

Hidden layers

Output layer



Computational graph

$$x \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y} \rightarrow L$$

$$z_1 = w_1 x$$

$$a_1 = \sigma(z_1)$$

$$z_2 = w_2 a_1$$

$$\hat{y} = z_2$$

$$L = \frac{1}{2}(\hat{y} - y)^2$$

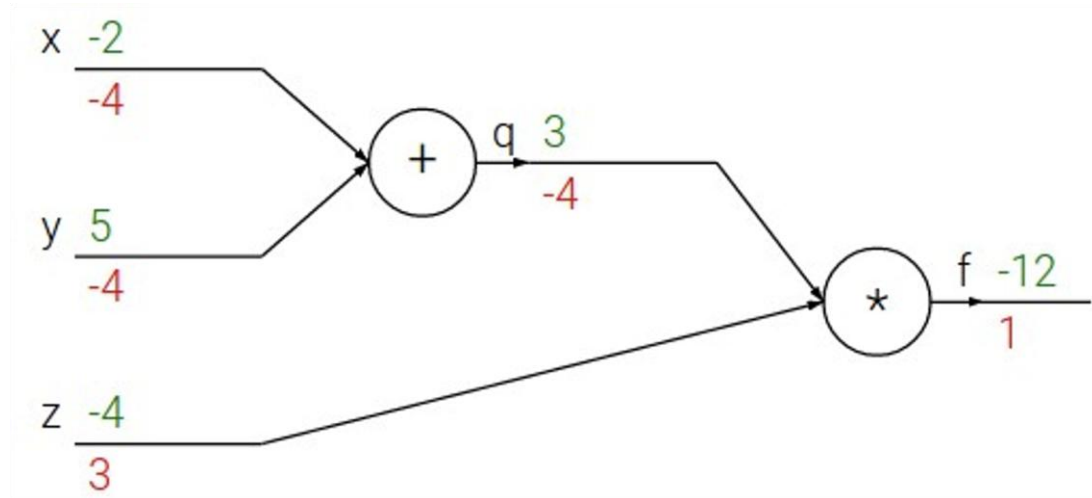
Now we want gradients:

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Computational graph

$$f(x, y, z) = (x + y)z$$



Forward pass: compute output

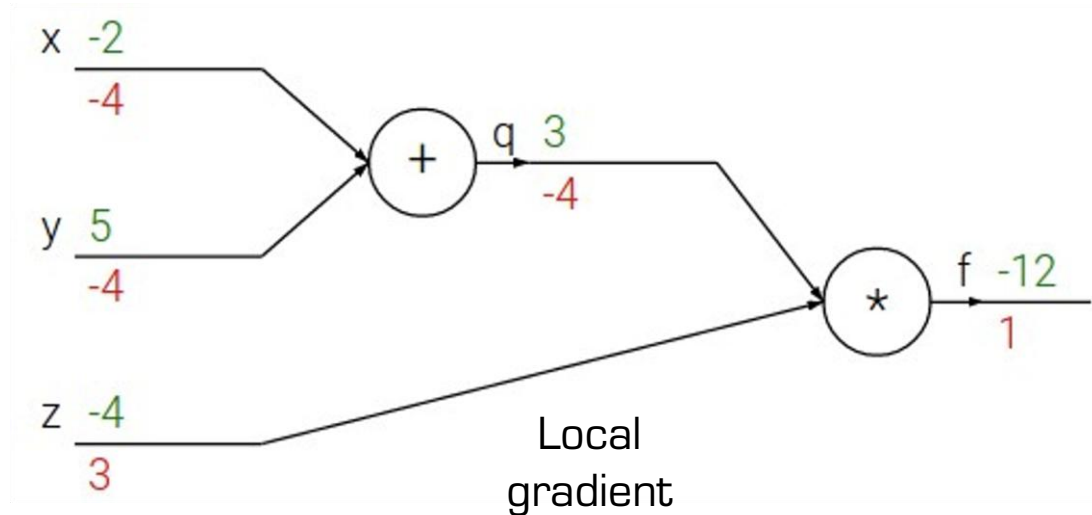
$$q = x + y \quad f = qz$$

Backward pass: compute gradient

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

Back propagation in a computational graph

$$f(x, y, z) = (x + y)z$$



Forward pass: compute output

$$q = x + y \quad f = qz$$

Backward pass: compute gradient

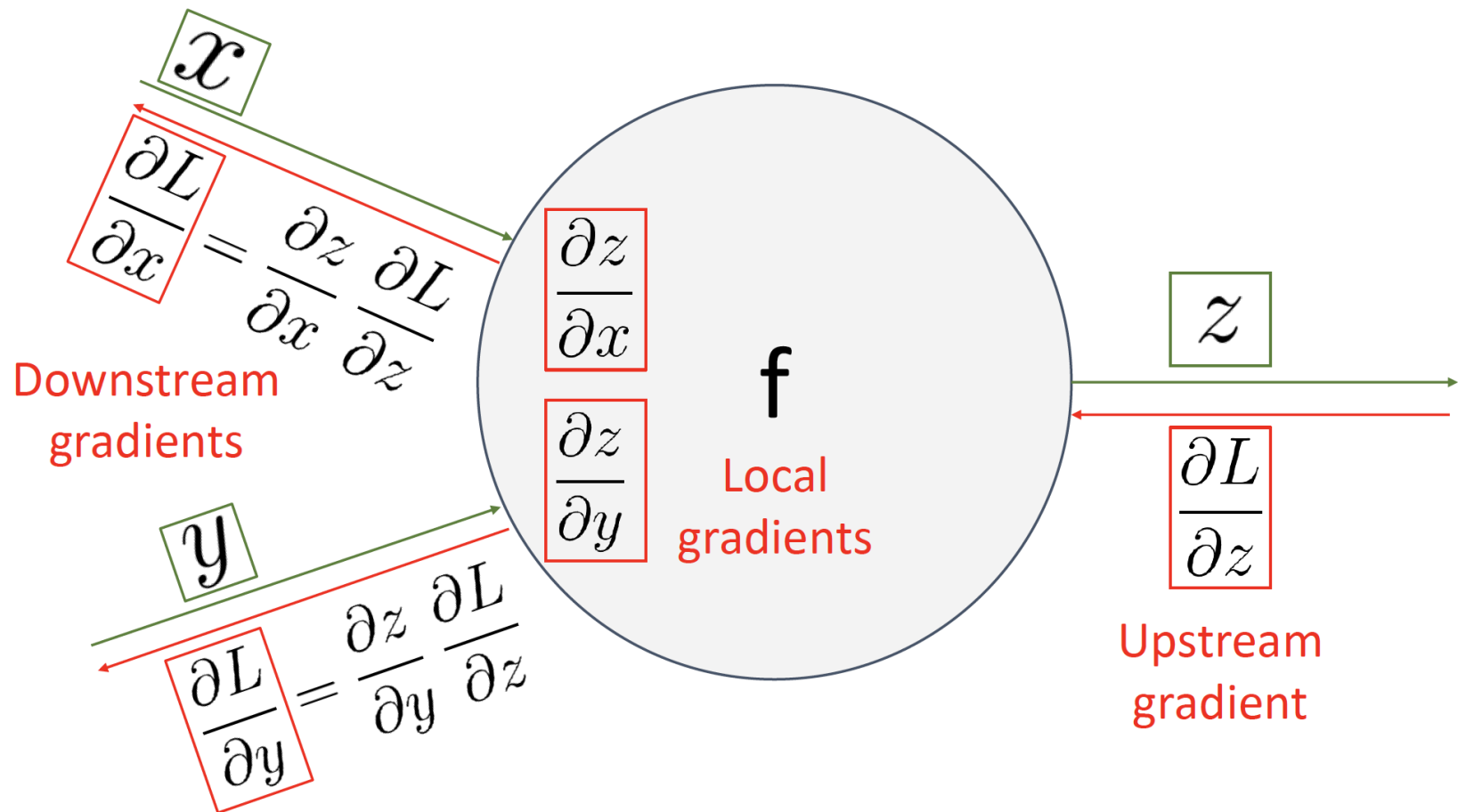
$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

$$\boxed{\frac{\partial f}{\partial y}} = \boxed{\frac{\partial q}{\partial y}} \boxed{\frac{\partial f}{\partial q}}$$

Downstream
gradient

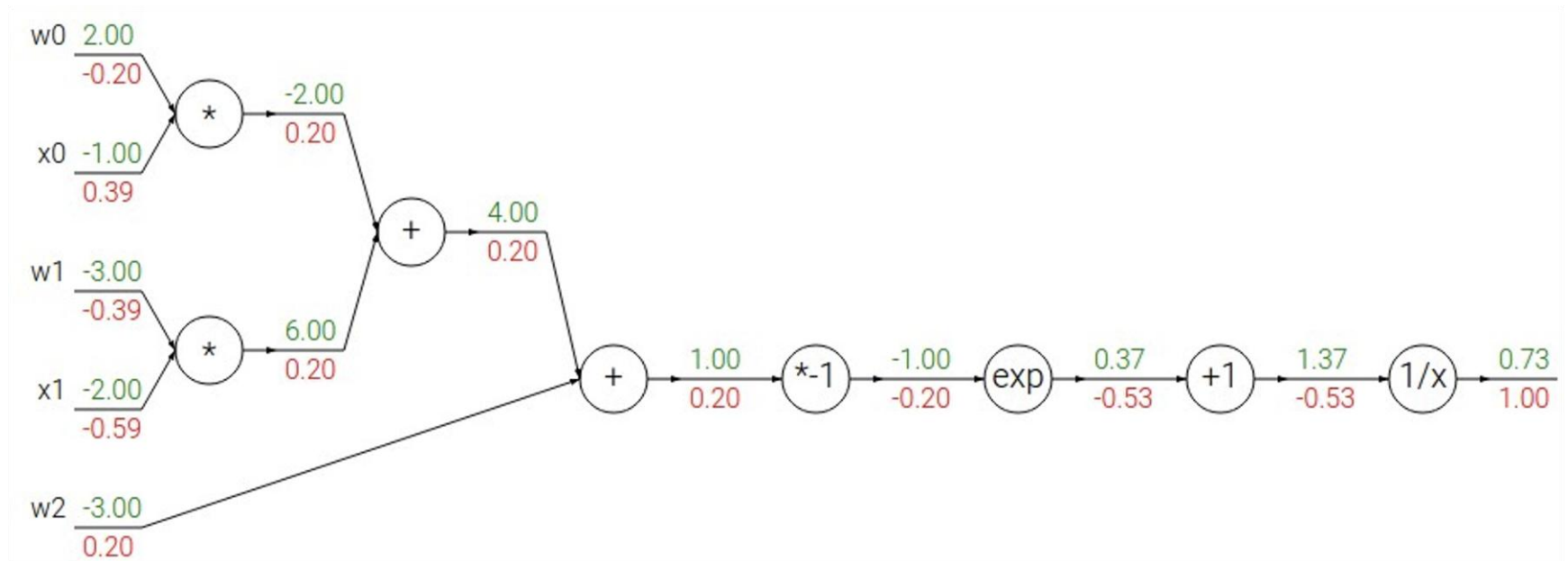
Upstream
gradient

Back propagation in a computational graph



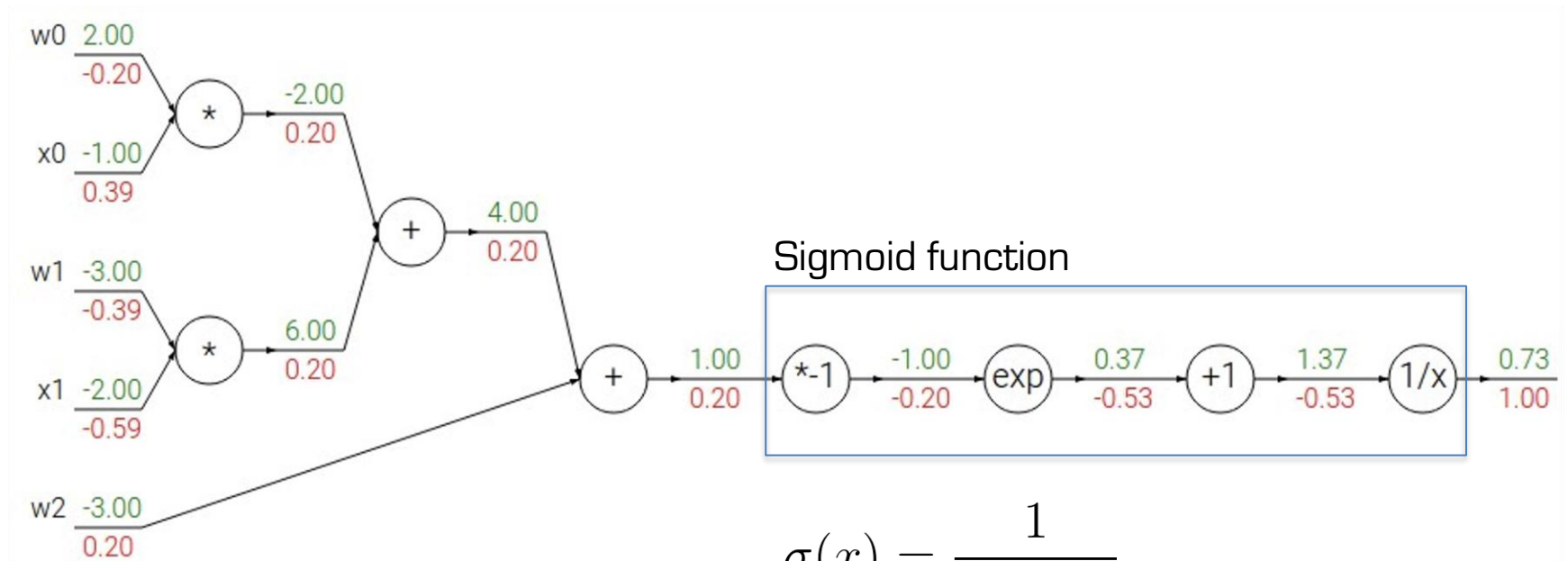
Back propagation in a computational graph

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



Back propagation in a computational graph

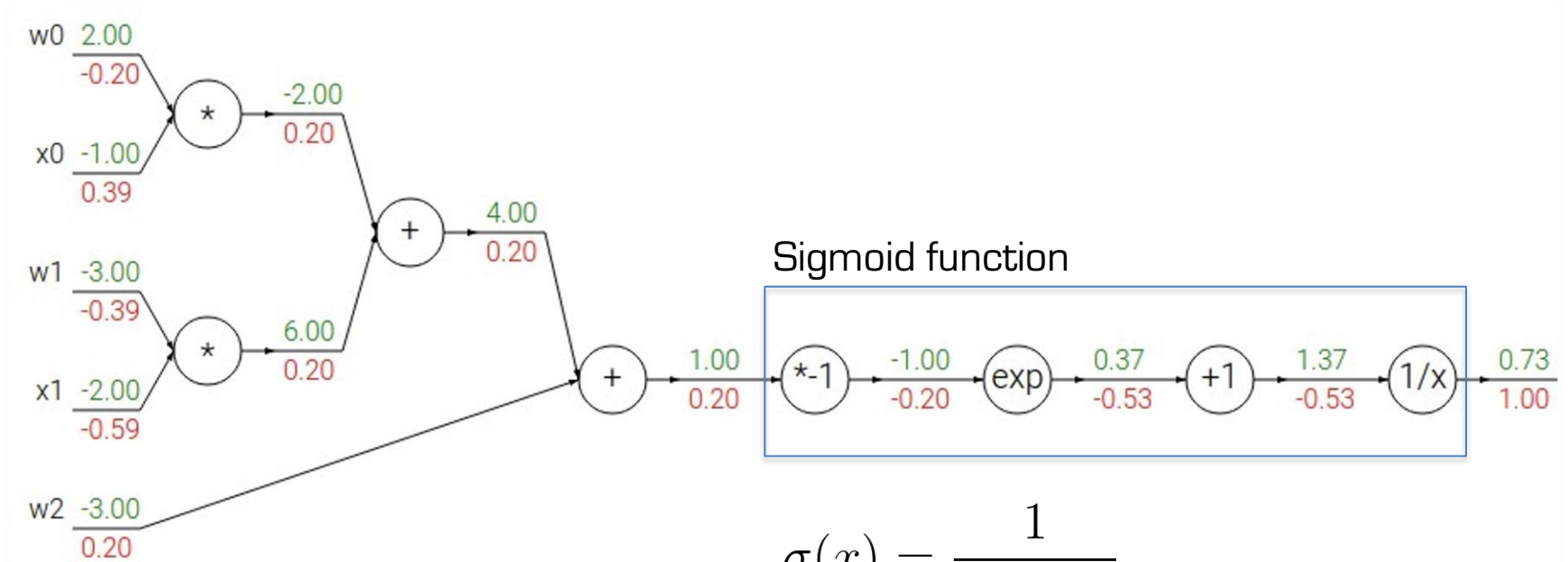
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}}$$



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Back propagation in a computational graph

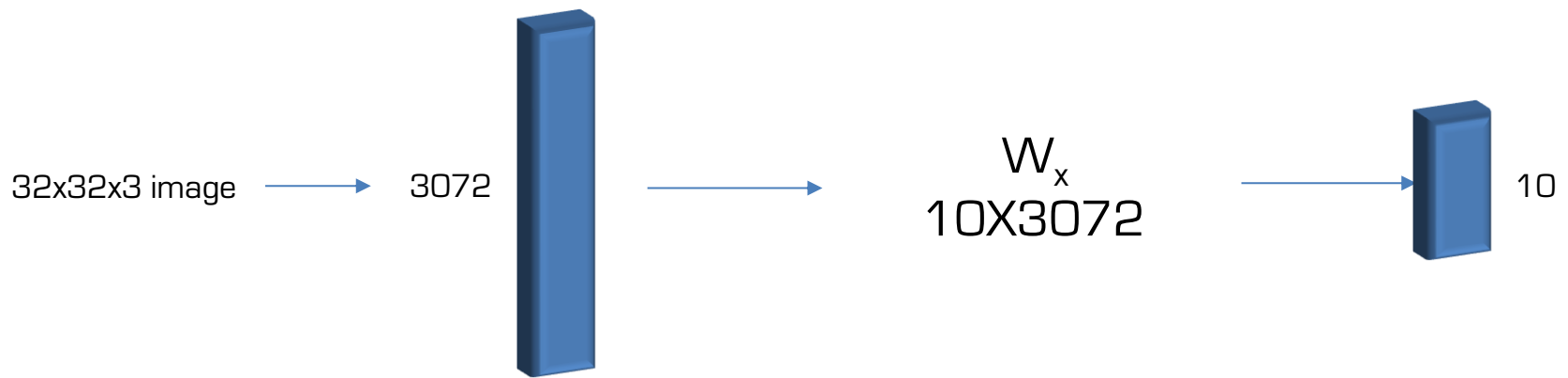
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

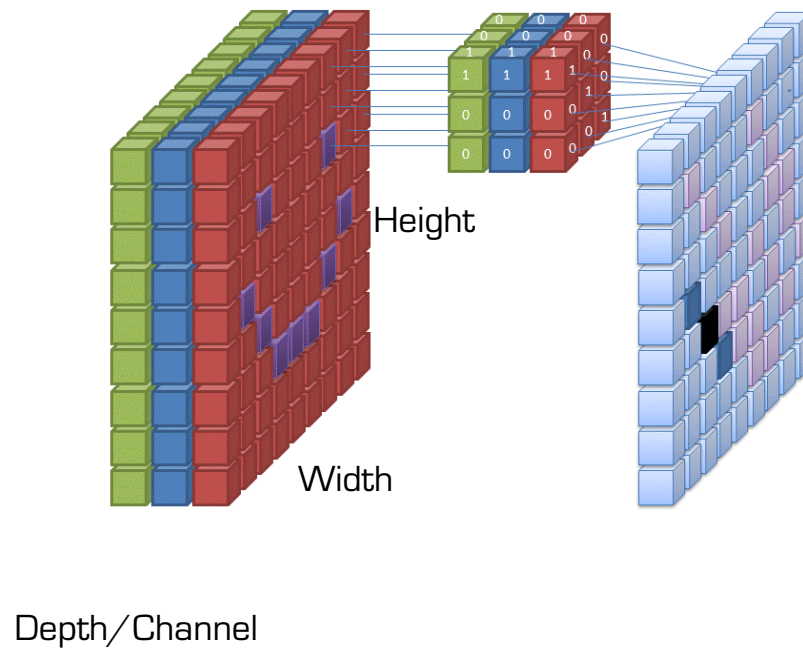
$$\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

Fully Connected Layer

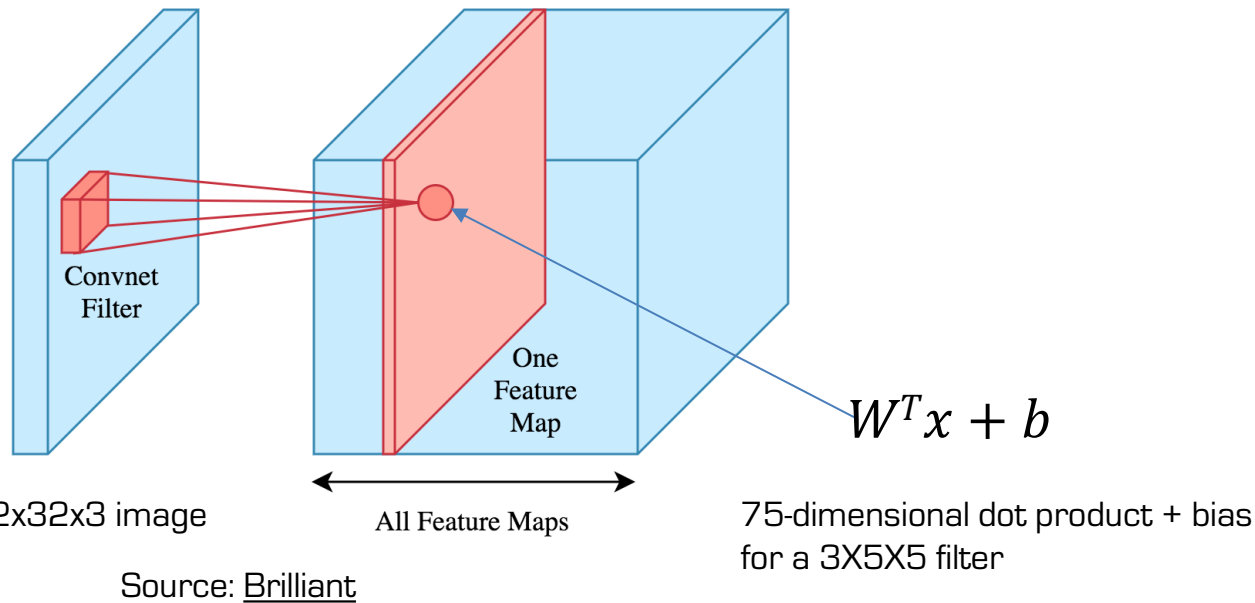


CIFAR 10

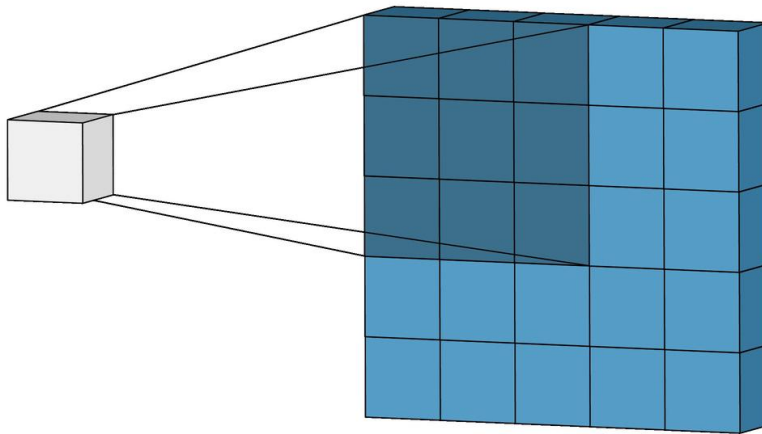
Convolution Layer



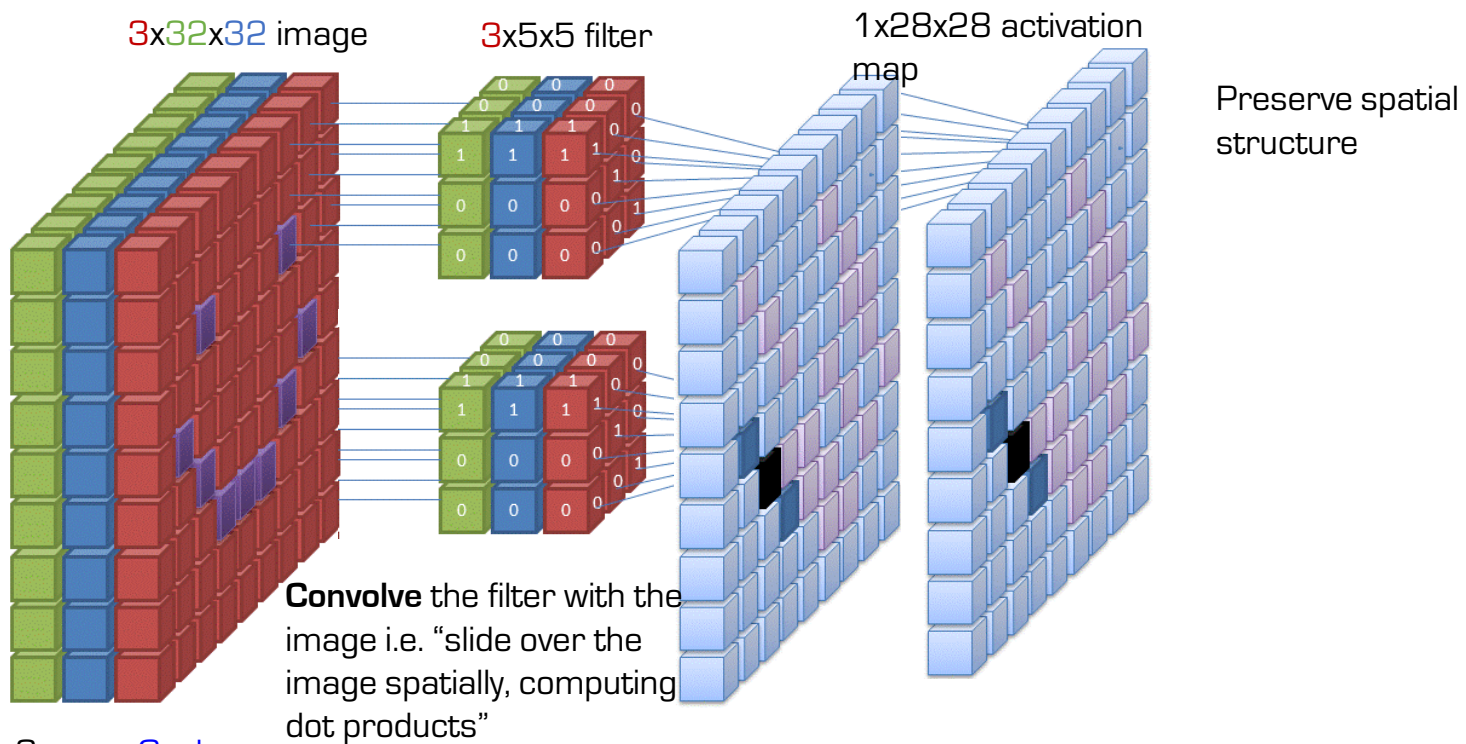
Convolution Layer



Convolution Layer

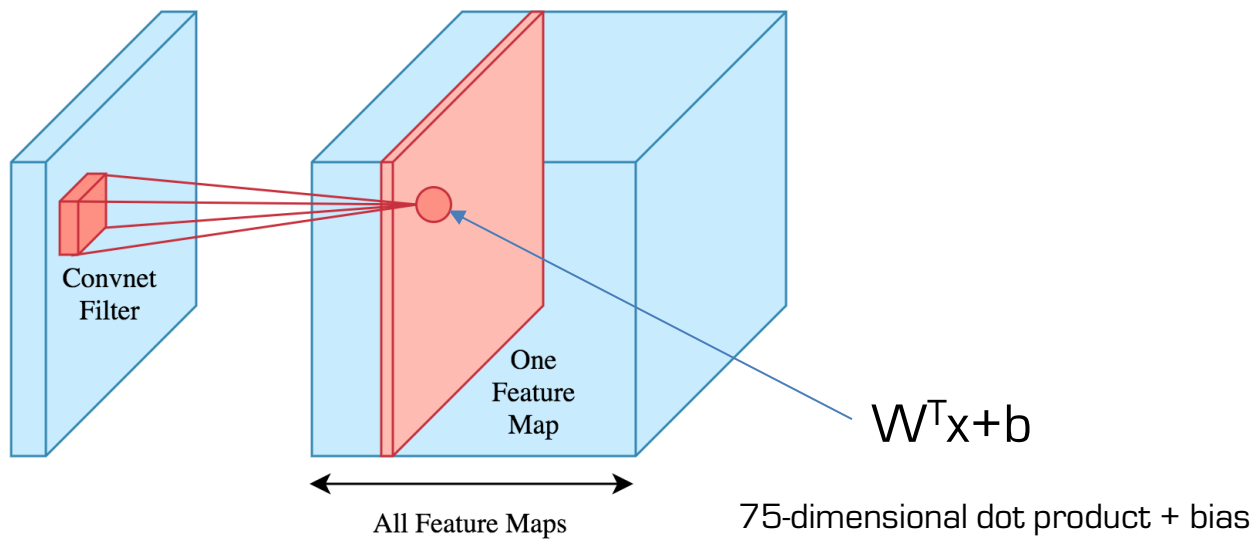


Convolution Layer



Source: [Cecbur](#)

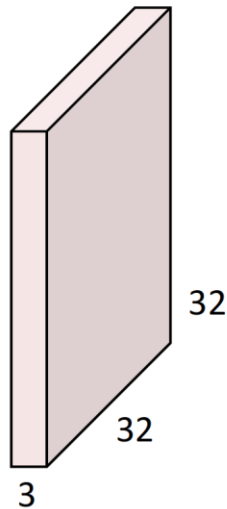
Convolution Layer



Source: [Brilliant](#)

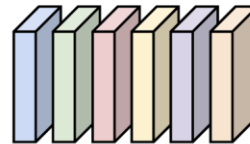
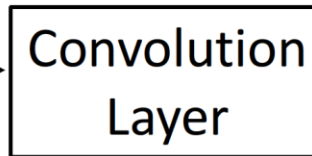
Convolution Layer

3x32x32 image

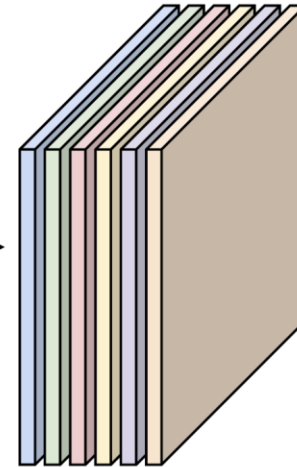


Consider 6 filters,
each 3x5x5

6x3x5x5
filters



6 activation maps,
each 1x28x28



Stack activations to get a
6x28x28 output image!

Slide by Justin Johnson

Convolution Layer

Input

$$N \times C_{\text{in}} \times H \times W$$

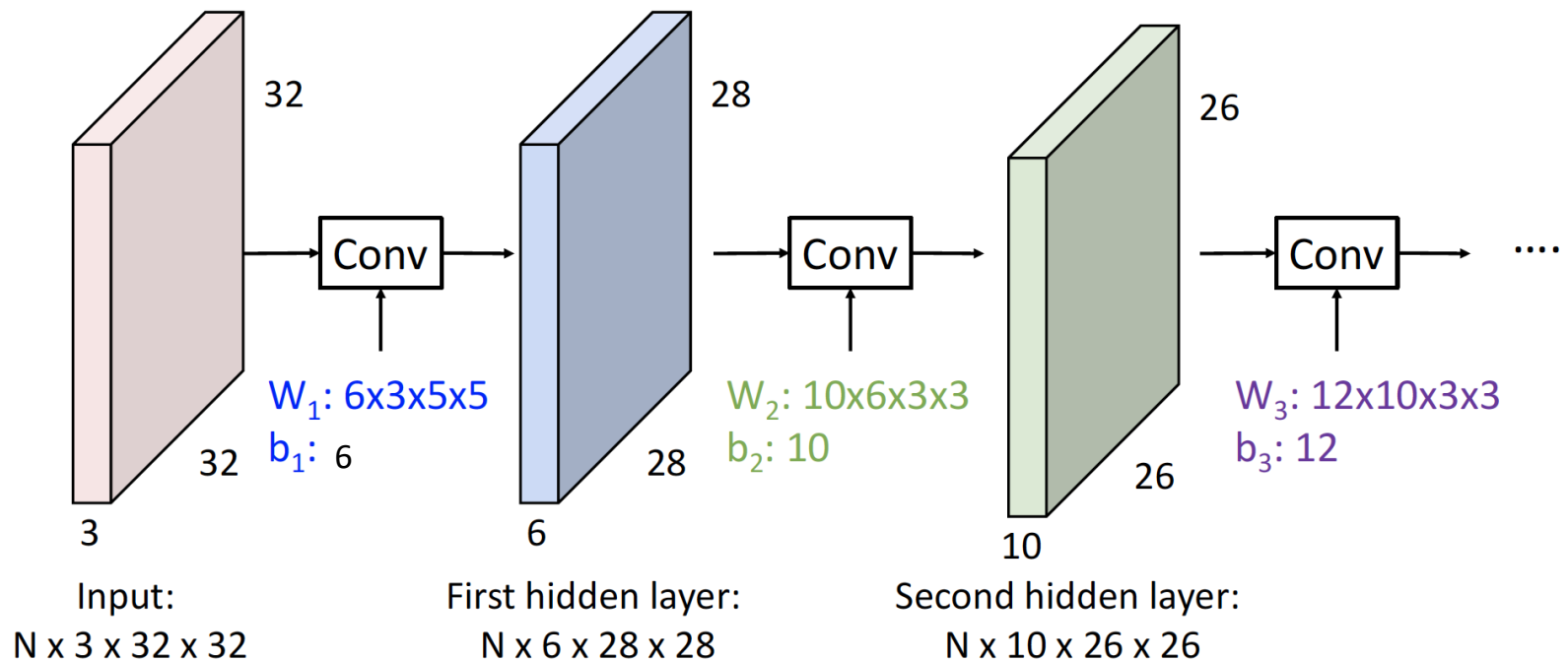
Filter

$$C_{\text{out}} \times C_{\text{in}} \times K_H \times K_W$$

Output

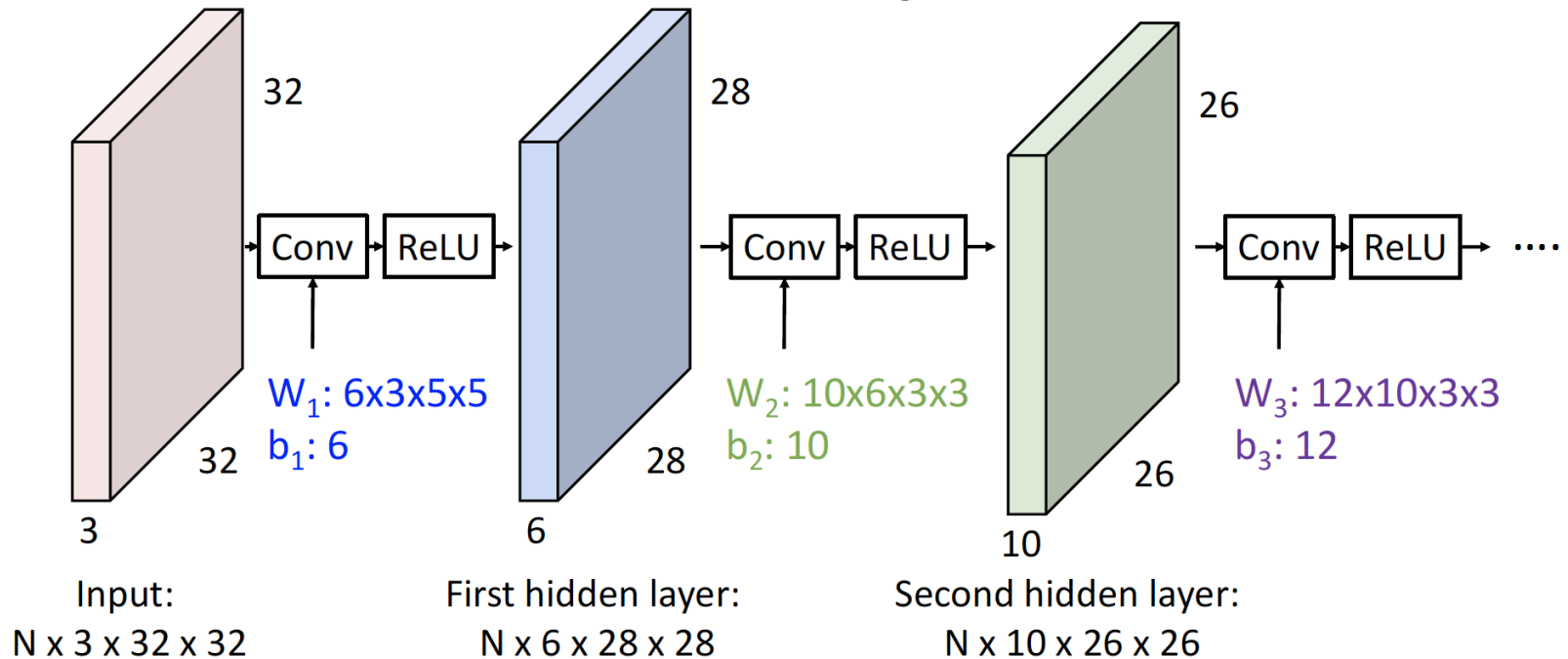
$$N \times C_{\text{out}} \times H' \times W'$$

Convolution Layer



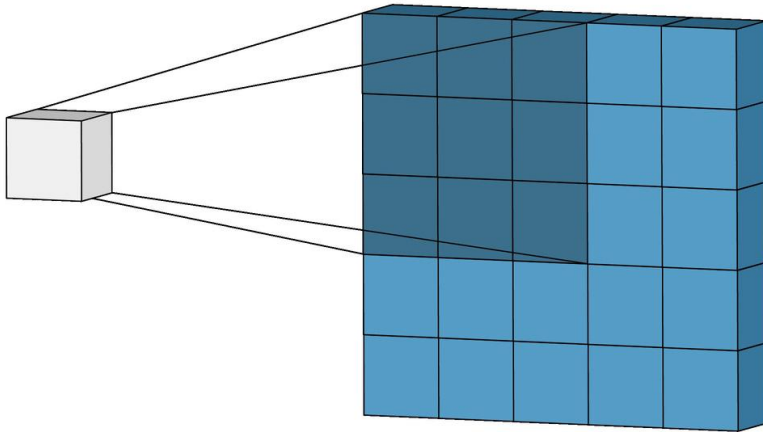
Slide by Justin Johnson

Convolution Layer



Slide by Justin Johnson

Convolution Layer



Any Problem with this?

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7

Filter: 3x3

Output: 5x5

In general: Problem: Feature
 Input: W maps “shrink”
 Filter: K with each layer!
 Output: $W - K + 1$

Solution: **padding**
 Add zeros around the input

Slide by Justin Johnson

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7

Filter: 3x3

Output: 5x5

In general:

Input: W

Filter: K

Padding: P

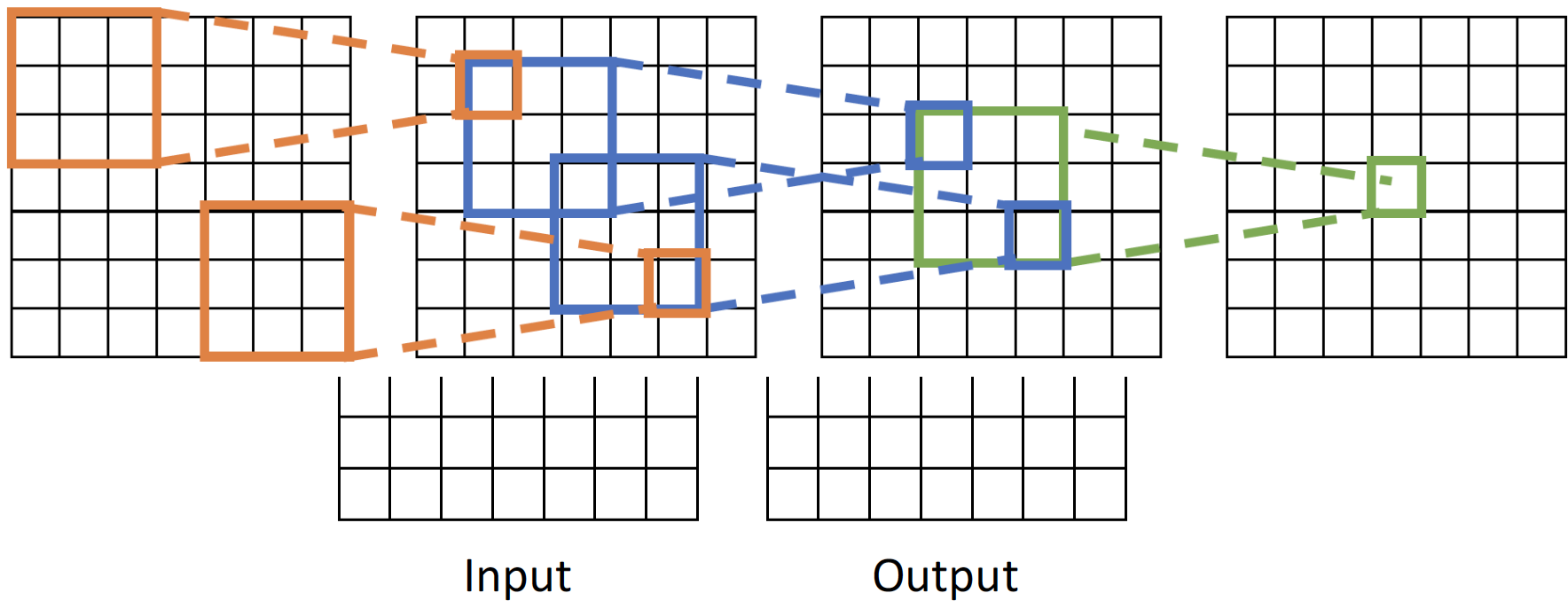
Output: $W - K + 1 + 2P$

Very common:

Set $P = (K - 1) / 2$ to
make output have
same size as input!

Slide by Justin Johnson

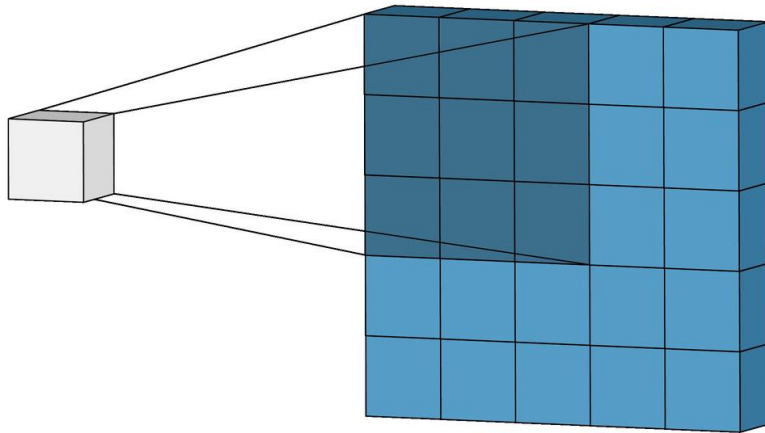
Receptive Fields



Slide by Justin Johnson

Solution: Downsample inside the network

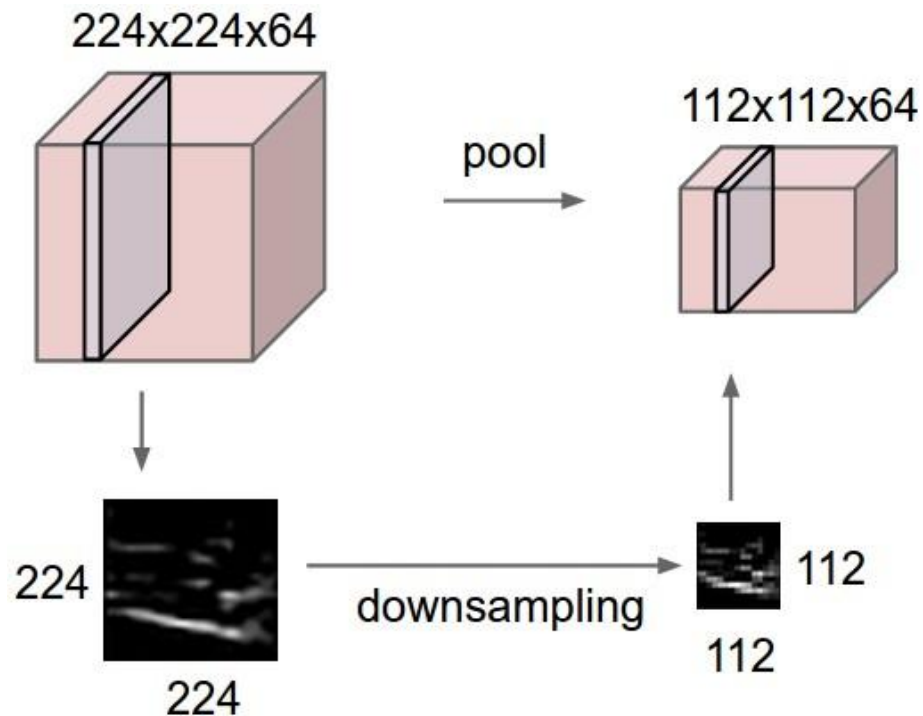
Convolution Layer



Source

Strided Convolution

Pooling Layers



Hyperparameters:

Kernel Size

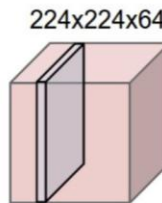
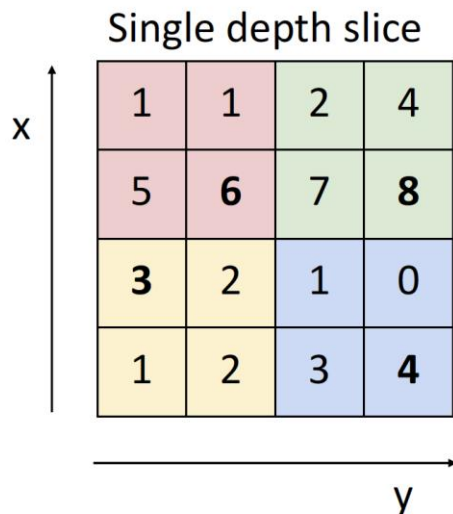
Stride

Pooling function

Slide by Justin Johnson

Pooling Layers

Max Pooling



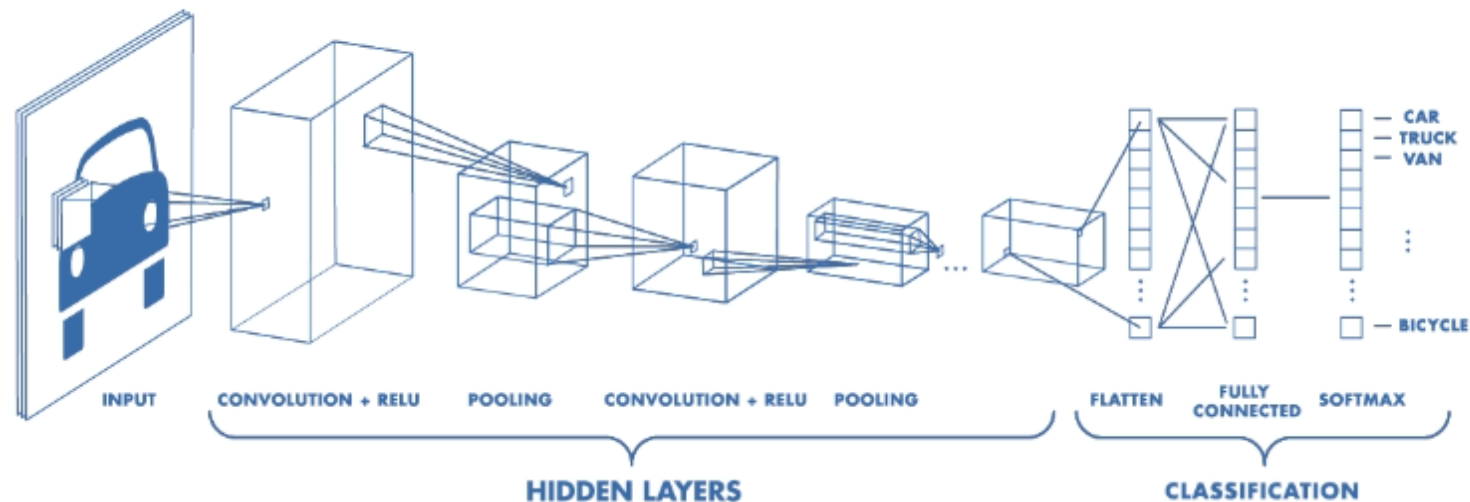
Max pooling with 2x2
kernel size and stride 2

6	8
3	4

Introduces **invariance** to
small spatial shifts
No learnable parameters!

Slide by Justin Johnson

Convolutional Networks



Source